

Exp. 8 Uploading Architecture Diagram (MVC & Microservices) in Azure DevOps Wiki

DEFINITION:

An Architecture Diagram represents the overall structure of a system, showing components, their relationships, and how they interact.

MVC (Model-View-Controller) and Microservices architectures are commonly used to design scalable and maintainable applications.

PURPOSE:

- To represent system architecture clearly
- To show separation of concerns using MVC
- To illustrate distributed system design using Microservices
- To understand component interaction and data flow

BASIC COMPONENTS (Notations):

1. Model (MVC)

- Represents data and business logic
- Handles database interactions

Example: User Model, Product Model

2. View (MVC)

- Represents the UI layer
- Displays data to users

Example: Web page, Mobile UI

3. Controller (MVC)

- Handles user input and requests
- Connects Model and View

Example: Login Controller

4. Microservices

- Independent services performing specific tasks
- Communicate via APIs

Example: Auth Service, Payment Service

5. API Gateway

- Entry point for all client requests
- Routes requests to appropriate services

6. Database

- Stores data for services
- Can be separate for each microservice

7. Communication

- Interaction between services
- REST APIs / HTTP / Messaging

HOW TO UPLOAD ARCHITECTURE DIAGRAM

AIM:

To upload an Architecture Diagram (MVC & Microservices) in the Wiki using Azure DevOps.

TOOLS REQUIRED:

- Azure DevOps
- Web browser
- Architecture Diagram image (PNG/JPG)

PROCEDURE:

Step 1: Open Azure DevOps

- Go to Azure DevOps
- Sign in with your Microsoft account

Step 2: Select Project

- Choose your existing project from the dashboard

Step 3: Navigate to Wiki

- Click on Wiki from the left menu
- If Wiki is not created:
 - o Click Create Wiki
 - o Select Project Wiki

Step 4: Create New Page

- Click on New Page
- Enter Architecture Diagram

Step 5: Upload Diagram

- Click on Insert Image (image icon)
- Choose Upload from Computer
- Select the Architecture Diagram file
- Click Upload

Step 6: Save the Page

- Click Save

Step 7: Verify Upload

- Ensure the Architecture Diagram is displayed correctly

RESULT:

Thus, the Architecture Diagram using MVC and Microservices was successfully uploaded to the Wiki using Azure DevOps.

Project Page

MB

Mapping government schemes to the beneficiaries

Private

Invite

About this project

Help others to get on board!
Describe your project and make it easier for other people to understand it.

Add Project Description

Project stats

Period: Last 7 days

Boards

0 Work items created

0 Work items completed

Repos

0 Pull requests opened

3 Commits by 1 authors

Pipelines

100%

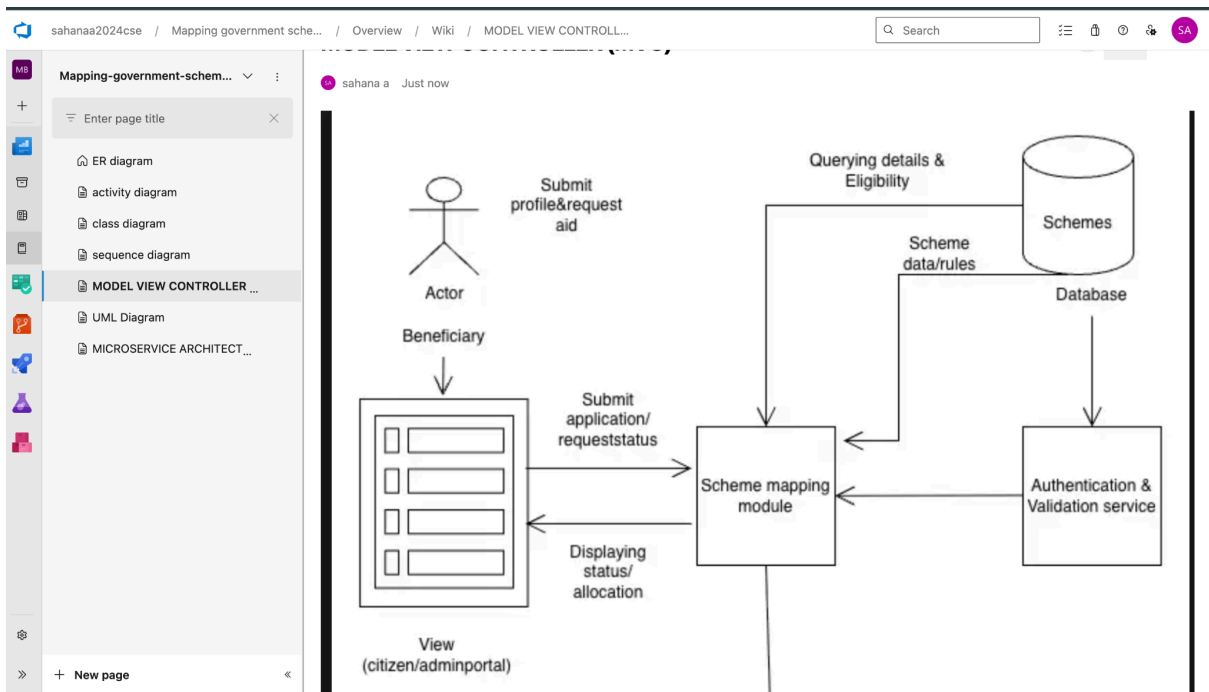
Builds succeeded

Members

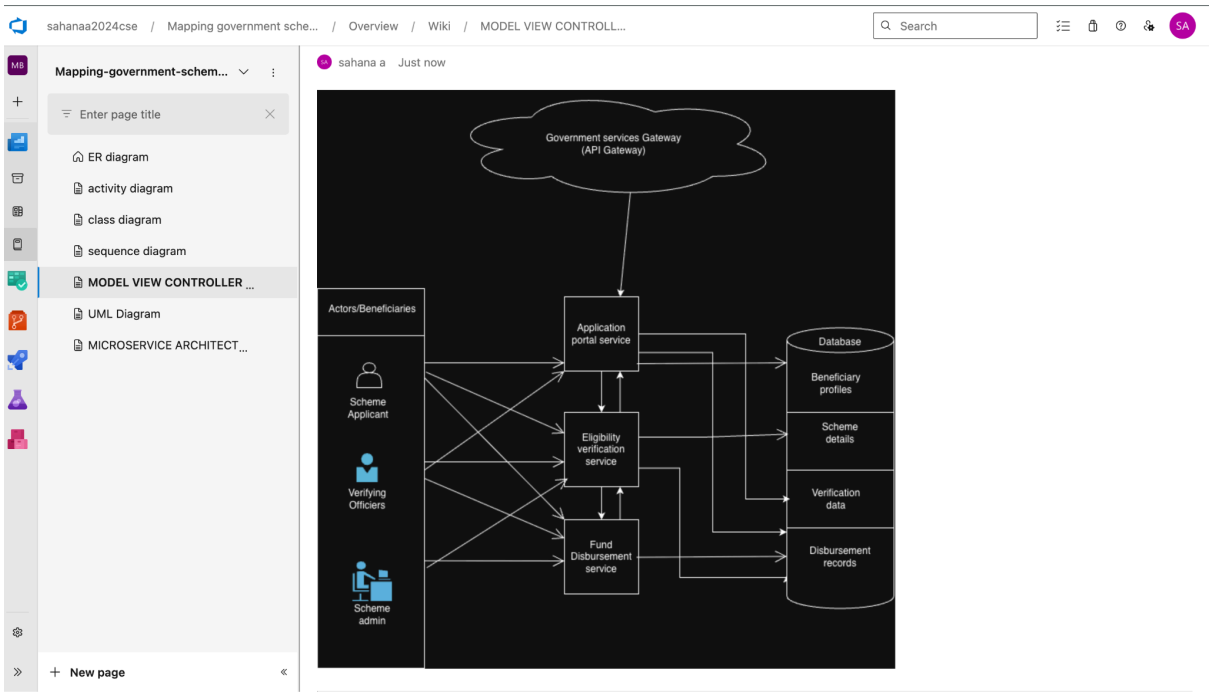
4

AP SS SA AM

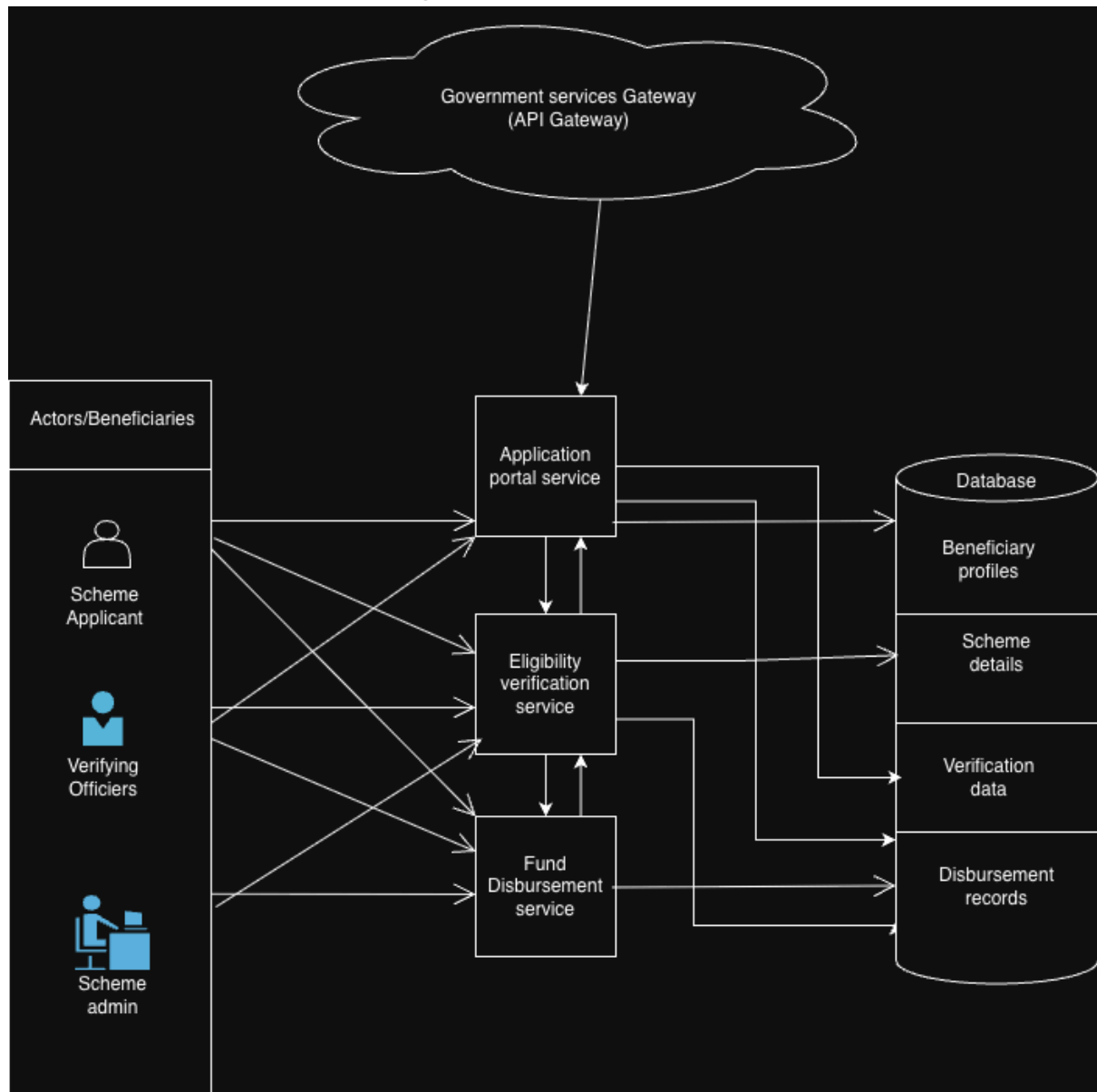
Wiki Page



MVC Architecture Diagram



Microservice Architecture Diagram



Mapping-government-schem...

+

Enter page title

×

ER diagram

activity diagram

class diagram

sequence diagram

MODEL VIEW CONTROLLER ...

UML Diagram

MICROSERVICE ARCHITECT...

+ New page

MODEL VIEW CONTROLLER (MVC)

sahana a

Just now

Unfollow

1

Edit

Micro Service Architecture Diagram

Government Service Gateway
(API Gateway)

Across beneficiary

Scheme applicant

Verifying officer

Scheme Admin

Application Portal Service

Eligibility Verification Service

Fund

Database

beneficiary Profiles

Scheme Details

Verification Data

Distribution of rewards

Micro Service Architecture Diagram

```
graph TD; Gateway([Government Service Gateway  
(API Gateway)]) <--> APS[Application Portal Service]; Gateway <--> EVS[Eligibility Verification Service]; Gateway <--> FDS[Fund Distribution Service]; APS <--> EVS; EVS <--> FDS; APS <--> DB[(Database)]; EVS <--> DB; FDS <--> DB; DB --> APS; DB --> EVS; DB --> FDS; APS --> AB[Across beneficiary]; APS --> SA[Scheme applicant]; EVS --> VO[Verifying officer]; FDS --> SA; FDS --> SAAdmin[Scheme Admin];
```

The diagram illustrates a Micro Service Architecture. At the top, a cloud-shaped box represents the **Government Service Gateway (API Gateway)**. Below it, three service boxes are arranged vertically: **Application Portal Service**, **Eligibility Verification Service**, and **Fund Distribution Service**. These services are interconnected with each other and with a **Database** on the right. The database contains four data stores: **beneficiary Profiles**, **Scheme Details**, **Verification Data**, and **Distribution of rewards**. On the left, four client roles are listed: **Across beneficiary**, **Scheme applicant**, **Verifying officer**, and **Scheme Admin**. Arrows indicate the flow of data and requests between these components.